CHEAT SHEET · GROUPE 1

HTTP en pratique

Méthodes · codes de statut · en-têtes · cache · exemples cURL | Sources : RFC 9110, RFC 9111, MDN, httpstatuses.io

MÉTHODES HTTP

Méthode	Rôle	Sûre	Idem.
GET	Lire une ressource	Oui	Oui
POST	Créer / traiter	Non	Non
PUT	Remplacer tout	Non	Oui
PATCH	Modifier en partie	Non	Souvent
DELETE	Supprimer	Non	Oui
HEAD	GET sans corps	Oui	Oui
OPTIONS	Options / CORS	Oui	Oui

CRUD : GET=SELECT · POST=INSERT · PUT/PATCH=UPDATE · DELETE=DELETE.

Idempotence : rejouer l'appel ne change pas le résultat. `POST` → `Idempotency-Key`.

CODES 2xx / 3xx

200 OK	Lecture / MAJ réussie
201 Created	Création + `Location`
202 Accepted	Traitement asynchrone
204 No Content	Succès sans corps
206 Partial	Téléchargement partiel
301/308 Moved	Déplacement définitif
302/307 Found	Redirection temporaire
304 Not Modified	Cache : copie valide

CODES 4xx — ERREUR CLIENT

400 Bad Request	Requête mal formée
401 Unauthorized	Auth absente / invalide
403 Forbidden	Authentifié, non autorisé
404 Not Found	Ressource inexistante
405 Method Not Allowed	Verbe non supporté
409 Conflict	Conflit d'état / doublon
415 Unsupported Media	Content-Type non géré
422 Unprocessable	Validation métier échouée
429 Too Many Requests	Quota ; `Retry-After`

CODES 5xx — ERREUR SERVEUR

500 Internal Error	Bug / exception non gérée
501 Not Implemented	Fonction non supportée
502 Bad Gateway	Réponse invalide d'un amont
503 Unavailable	Maintenance / surcharge
504 Gateway Timeout	Amont trop lent

5xx : retenter avec backoff + jitter. **4xx :** corriger la requête.

CACHE & CONDITIONNEL

- `Cache-Control: max-age=60, public` — cache 60 s.
- `no-store` — jamais de cache (données sensibles).
- `ETag: "v42"` + `If-None-Match` → **304** si inchangé.
- `Last-Modified` + `If-Modified-Since`.
- `Vary: Accept, Authorization` — clé de cache.
- **GET** cacheable ; **POST/PUT/PATCH/DELETE** non par défaut.

EN-TÊTES CLÉS

Requête	Rôle
`Host`	Serveur cible (HTTP/1.1)
`Accept`	Format de réponse voulu
`Content-Type`	Format du corps envoyé
`Authorization`	`Bearer <token>`
`Idempotency-Key`	Éviter les doublons
`If-None-Match`	Cache conditionnel

Réponse	Rôle
`Content-Type`	Type MIME du corps
`Location`	URI créée (201)
`Cache-Control`	Politique de cache
`ETag`	Version de la ressource
`Retry-After`	Délai après 429/503
`X-Request-Id`	Corrélation / support

REQUÊTE / RÉPONSE

```
POST /api/v1/tasks HTTP/1.1
Host: api.exemple.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJ...

{"title":"Réviser HTTP","due_date":"2026-11-05"}

HTTP/1.1 201 Created
Location: /api/v1/tasks/43
Content-Type: application/json

{"id":43,"title":"Réviser HTTP","done":false}
```

EXEMPLES cURL

```
# Lister avec filtre et pagination
curl -G https://api.exemple.com/v1/tasks \
  -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "done=false" \
  --data-urlencode "limit=20"

# Créer
curl -X POST https://api.exemple.com/v1/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title":"Réviser HTTP"}'

# Modifier partiellement
curl -X PATCH .../tasks/42 \
  -H "Content-Type: application/json" \
  -d '{"done":true}'

# Supprimer
curl -X DELETE .../tasks/42 -i
```

ERREURS NORMALISÉES (RFC 9457)

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{"type":"https://api.exemple.com/errors/validation",
 "title":"Échec de validation","status":422,
 "detail":"Le champ 'due_date' est invalide.",
 "instance":"/api/v1/tasks","request_id":"9b2f7c1e"}
```

- Un format unique pour toute l'API.
- Code HTTP correct, jamais 200 + erreur.
- Message utile, sans stack trace ni secret.
- `request_id` dans les logs et la réponse.

CHECKLIST

- HTTPS partout ; HSTS activé.
- Valider types, tailles, formats, listes blanches.
- Pagination + limites de taille + quotas.
- Timeouts et retries avec backoff.
- Logs corrélés (`X-Request-Id`).
- CORS : origines explicites, pas `*` en prod.
- Ne pas exposer de détails internes dans les erreurs.

CODES DÉTAILLÉS 1xx / 2xx / 3xx

1xx	2xx	3xx
100 Continue	**200** OK	**301** Moved Permanently
101 Switching Protocols	**201** Created	**302** Found
103 Early Hints	**202** Accepted	**304** Not Modified
	204 No Content	**307** Temporary Redirect
	206 Partial Content	**308** Permanent Redirect

CORS

- **Preflight :** `OPTIONS` + `Access-Control-Request-Method` .
- **Réponse :** `Access-Control-Allow-Origin` , `-Methods` , `-Headers` .
- **Credentials :** origine explicite, jamais `*` .
- **Max-Age :** mise en cache du preflight.

SÉCURITÉ HTTP

- `Strict-Transport-Security` (HSTS).
- `X-Content-Type-Options: nosniff` .
- `Content-Security-Policy` pour les interfaces.
- `Referrer-Policy` , `X-Frame-Options` .

CLIENT HTTP ROBUSTE

- **Timeouts** courts et explicites.
- **Retries** uniquement 429/5xx, backoff + jitter.
- **Circuit breaker** pour les dépendances.
- **Idempotency-Key** pour les écritures.
- **Compression :** `Accept-Encoding: gzip, br` .

STREAMING & TEMPS RÉEL

- **Chunked :** transfert progressif sans `Content-Length` .
- **SSE :** flux serveur → client, simple HTTP.
- **WebSocket :** bidirectionnel, `101 Switching Protocols` .
- **HTTP/2 :** multiplexage ; **HTTP/3 :** QUIC/UDP.